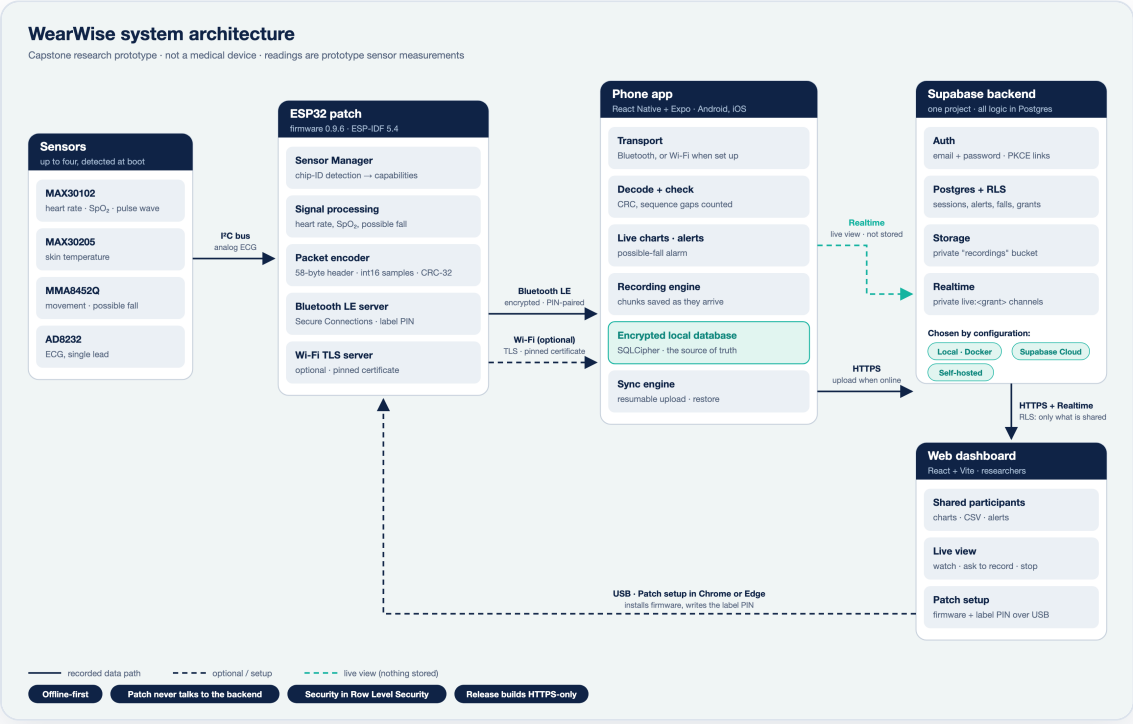
WearWise

HANDOVER DOCUMENTATION

Handing over WearWise

What is being handed over, how WearWise works, how to run, release and change it, and what the new owner needs to do and decide.



Handover release v1.0.0 · app 0.1.0 · firmware 0.9.6

October 2026 · WearWise Project Team

Zakiul Atheef · Hari Bethelli · Siri Bethelli · Thevindu Kodagoda Munasinghage

Research prototype.
Not a medical device.

Contents

PART 1 · THE HANDOVER AT A GLANCE

1 Summary 3
2 What is handed over 3
3 State and verification 4
4 Project owner and team 5
5 The new owner's first steps 5

PART 2 · HOW WEARWISE WORKS

6 System overview 6
7 Key design decisions 7
8 Phone app 7
9 Firmware and sensors 8
10 Bluetooth protocol 8
11 Backend, accounts and security 9

PART 3 · RUNNING AND RELEASING

12 Local development 9
13 Tests 9
14 Administration 10
15 Cloud deployment 10
16 Release builds 11

PART 4 · CHANGING WEARWISE

17 Adding a sensor or a database field 11
18 Maintenance and responsibilities 12

PART 5 · LIMITATIONS AND NEXT STEPS

19 Known limitations 12
20 Future improvements 13
A Documentation map 13
B Repository layout 14

PART 1

The handover at a glance

What is being handed over, its state, who built it, and the new owner's first steps.

1 Summary

WearWise is a **capstone research prototype** for collecting wearable sensor data with the participant in control. It has four parts:

- an **ESP32 sensor patch** that measures heart rate, SpO_2, pulse wave, skin temperature, movement and ECG;
- a **phone app** (Android and iOS) that pairs with the patch, records sessions into an encrypted database on the phone and backs them up;
- a **Supabase backend**: accounts, the cloud copy of recordings, sharing, alerts and live view;
- a **web dashboard** where researchers and clinicians see what participants chose to share, and which installs firmware on patches.

Status

WearWise works end to end and has been tested on real hardware, but it is a **research prototype**. It is not a medical device, its measurements are not clinically validated, and fall detection reports a *possible fall* only. Its security practices are informed by relevant guidance; nothing about it is certified or independently tested.

Ready

Code, firmware, database, tests and documentation, published as one tagged release with a clean history.

Verified

Two clean dry runs of the documented setup, all test suites, 13 hardware tests, and a hosted-schema comparison.

Owner's turn

Accounts and secrets of your own, governance, and a few decisions before real participants (section 5).

2 What is handed over

Source code	One Git repository (currently the private GitHub repository `zackathef-Dev/wearwise`), branch `main`. The release is tagged `v1.0.0-handover`. It was published as a fresh history with no personal data, and contains no secrets.
Versions	Phone app 0.1.0 (Expo SDK 57, React Native 0.86) · web dashboard 0.1.0 (React 19, Vite) · patch firmware 0.9.6 (ESP-IDF 5.4) · database: 11 migrations (Postgres 17)

Firmware images	Bundled in the dashboard for Patch setup (`apps/dashboard/public/firmware/`), built from a clean checkout; `manifest.json` records the commit.
Hardware	**Not handed over**: the prototype patches stay with the team. Everything needed to build patches is included: the bill of materials and wiring (`docs/hardware.md`, Setup Guide Part 5) and the firmware, which installs from the dashboard's Patch setup page. Testing used three prototypes on firmware 0.9.6: WearWise-6280 (all four sensor modules), -6F84 (MAX30102 only) and -6990 (no sensors).
Hosted backend	The original capstone Supabase project. It matches the migrations (section 3). Hand it over deliberately or retire it; the owner normally starts their own project (section 15).
Documentation	The README and the `docs/` folder, plus five PDFs: this document, the User Guide, the Setup Guide, Adding a Researcher or Clinician, and the Security Documentation (Appendix A).
Licence	PolyForm Noncommercial License 1.0.0, © 2026 WearWise Project Team (`LICENSE`, `NOTICE`): free to use, copy and change for noncommercial purposes; selling it or other commercial use needs the team's written permission.

Not included: the prototype patch hardware and the design and presentation material (mock-ups, Figma, slides, trade-show and video material), which stay with the team; and any secrets, which the owner creates (section 18).

3 State and verification

Check (14 October 2026)	Result
New-developer dry run: setup from the README alone, twice, on a clean machine state	**Pass** every failure found in pass 1 was fixed in the docs or code; pass 2 ran clean
Database security suites on a real local Supabase stack	**Pass** 277 plain-SQL checks + 8 pgTAP
Unit tests and type checks	**Pass** app 283 tests · dashboard 110 · shared legal package 51
Two-account isolation through the public API	**Pass** 7/7 locally; 19/19 on the hosted project (13 October)
Hardware tests H1–H13 (Samsung phone, three patches): pairing, encryption, reconnect, recording and sync, pinned-TLS Wi-Fi, sharing, live view, remote recording, fresh PIN pairing	**Pass** 13/13
Hosted database compared with the migrations (read-only schema fingerprint)	**Match** in everything that affects behaviour
Published git history audit (keys, tokens, personal data, paths)	**Pass** none found
Dependency alerts (GitHub Dependabot)	8 of 13 fixed; the other 5 assessed: build tooling, not affected, or mitigated (`docs/security/dependency-advisories.md`)

Details: `docs/testing/handover-dry-run-2026-10-14.md`, `docs/testing/hardware-test-matrix.md`, `docs/testing/real-device-2026-10-13.md`, and the Security Documentation.

4 Project owner and team

WearWise was developed as part of the La Trobe University CSE3CAP Capstone Project.

Project Owner	**Shanmuga Sundar Dhanabalan**, Deputy Director, CTI, La Trobe University: provided the project brief and stakeholder direction

WearWise was designed and implemented by the **WearWise Project Team**, the student development team:

Zakiul Atheef	Project Lead, Scrum Master, Technical Lead, Security Engineering Lead and Integration Lead
Hari Bethelli	Cloud & Data Engineer
Siri Bethelli	UI/UX & Data Visualisation Engineer
Thevindu Kodagoda Munasinghage	Security Engineer, Documentation Lead and QA & Testing Lead

The team credits also appear in the app (Profile → Legal & privacy), the dashboard (Legal page and footer) and `TEAM.md`. We acknowledge the project owner and La Trobe University for providing the project brief, stakeholder direction, and feedback throughout the capstone project.

5 The new owner's first steps

1. **Take over the code.** Have the GitHub repository transferred to your organisation (or be added as an owner), clone it, and check out `v1.0.0-handover`.
2. **Run it locally.** Follow the **Setup Guide** PDF or `README.md`: local Supabase, dashboard, app on an emulator. About two hours the first time.
3. **Set up your own backend.** A Supabase project under your organisation's account, the migrations applied (never the demo seed), the project settings, and the first researcher account (section 15). Confirm it with the schema fingerprint.
4. **Decide on the original project.** Have the team add you as Owner of the Supabase organisation that holds it, or retire it. Former members revoke their access and personal tokens.
5. **Create your secrets.** Android release keystore, Apple signing, database password, SMTP credentials and access tokens, kept in your organisation's secret store. None are handed over (section 18).
6. **Build or source patches.** An ESP32 board and the sensor modules in `docs/hardware.md`; install the firmware with Patch setup, which writes the PIN to print on each patch's label. Use **New PIN** whenever a patch changes hands.

7. **Governance before participants.** Ethics approval, data region and retention, the organisation's details in `packages/legal/src/org.ts` (the legal documents show "draft" until then), and an independent security review.

8. **Make the open decisions** below.

Decisions for the owner

Decision	Why it matters
Lock patch hardware (Secure Boot, flash encryption)	**High** A patch's flash is unencrypted: brief USB access can copy its pairing keys and PIN. The fix is permanent per chip and needs a signed-update process. Decide before patches go to participants.
Two-factor sign-in for staff	**Medium** One leaked researcher password exposes every participant sharing with that person.
CAPTCHA or enrolment code at sign-up	**Medium** Open sign-up lets scripted accounts fill storage.
Hosted project settings	**Medium** Email confirmation, SMTP, Site URL and redirects, Realtime public access off.
Retention policy	**Medium** No automatic retention or purge exists; it must match the ethics approval.

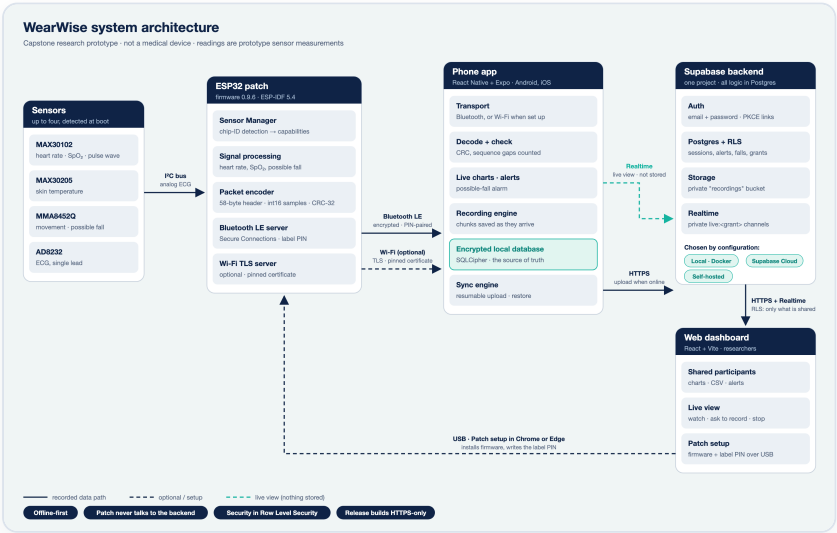
The full risk register is in the Security Documentation, section 24.

PART 2

How WearWise works

The system, the decisions behind it, and each component.

6 System overview



WearWise architecture. The patch talks only to the phone; the phone records offline and backs up to Supabase; the dashboard reads only what participants share.

A participant wears the patch. The phone app pairs with it over Bluetooth using the PIN on its label, shows live readings, and records sessions into an encrypted database on the phone. When online, it uploads finished sessions to Supabase. A participant can create a one-time **share code**; a researcher redeems it in the dashboard and can then read the shared signals, see alerts and falls, and, while the participant allows it, watch **live view**. Either side can end the sharing.

Component	Path	Technology
Patch firmware	`firmware/wearwise-esp32/`	ESP32, ESP-IDF 5.4, C
Phone app	`apps/mobile/`	React Native 0.86, Expo SDK 57, TypeScript, SQLCipher
Web dashboard	`apps/dashboard/`	React 19, Vite, TypeScript
Backend	`supabase/`	Supabase: Postgres 17, Auth, Storage, Realtime
Shared packages	`packages/`	sensor-protocol (the firmware contract), legal, password, design-tokens

7 Key design decisions

1. **Offline-first, the phone is the source of truth.** Recording never depends on the network. The cloud is an upload-only mirror plus a restore path for a new phone.
2. **The patch has no backend knowledge.** It speaks only to the phone, so the backend can move without touching firmware.
3. **One binary protocol with a shared fixture.** Firmware and app test against the same `packages/sensor-protocol/fixtures/v1.json`.
4. **Capability discovery.** The firmware reports the sensors it found; nothing assumes a sensor is there.
5. **Security in Postgres (Row Level Security), not in the apps.** The publishable key is public by design.
6. **No custom server code.** RPCs, triggers and policies replace an API server; no Edge Functions and no server secrets of our own.
7. **Configuration selects the environment** (local, the owner's cloud, self-hosted). Release builds are HTTPS-only.

8 Phone app

React Native 0.86 + Expo SDK 57, TypeScript, expo-router, zustand. Layers: `app/` (screens), `features/` (state per feature), `domain/` (pure logic: recording engine, sync engine, fall detector, vitals), `data/` (Bluetooth, Wi-Fi, SQLCipher store, Supabase) and `platform/` (secure storage, notifications). It runs as a development build (`npx expo run:android|ios`), never Expo Go. A simulated patch exercises everything without hardware.

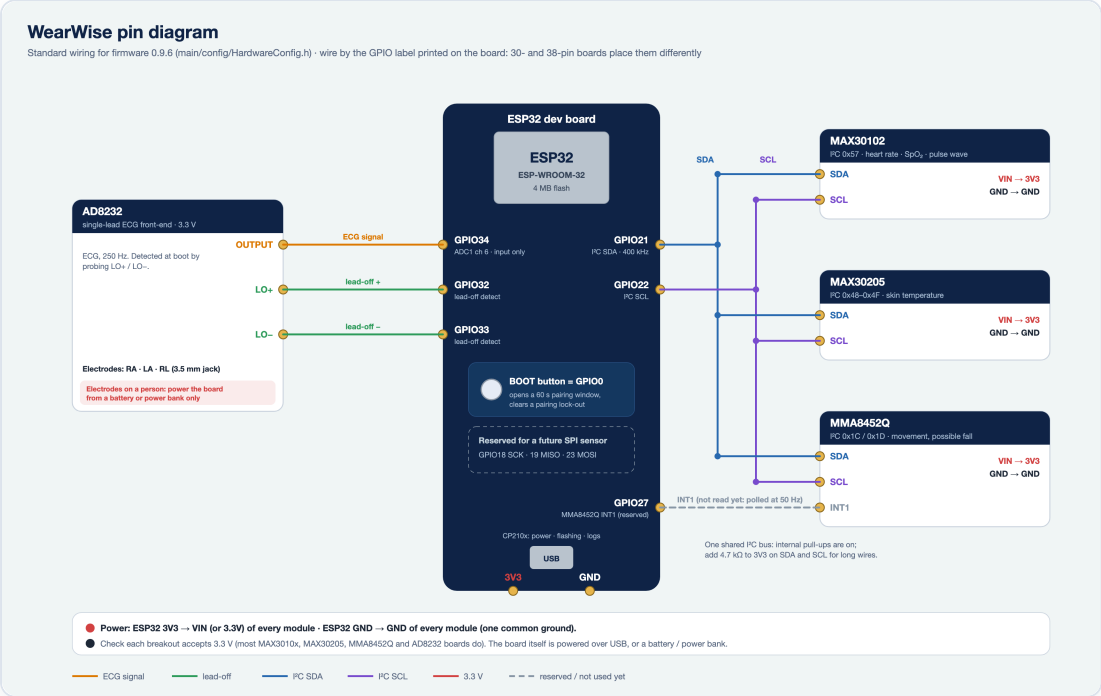
Local database: op-sqlite with SQLCipher, key in the OS secure store; inline migrations v1–v9 in `src/data/local/sqlite-session-store.ts`. Sample chunks are stored exactly as received, with CRC. Changing the schema means adding a migration step (never editing an old one) and a test.

Read: `docs/mobile-app.md`, `docs/architecture/offline-first.md`.

9 Firmware and sensors

ESP-IDF 5.4 C project in `firmware/wearwise-esp32/`. `SensorManager` scans I²C and verifies chip IDs; drivers implement one interface (`WearWiseSensor.h`). `gatt.c` is the Bluetooth service, `ble_pairing.c` the PIN and pairing rules, `ws_server.c` the optional Wi-Fi stream. The version is in `FirmwareConfig.h`. Patches are flashed without a toolchain from the dashboard's **Patch setup** page, which also writes the label PIN.

Sensor	Signals	Notes
MAX30102	Heart rate (1 Hz), SpO_2 (1 Hz), pulse wave (100 Hz)	On-device heart-rate pipeline; SpO_2 uncalibrated
MAX30205	Skin temperature (2 Hz)	
MMA8452Q	Movement magnitude (50 Hz)	The phone detects possible falls from it
AD8232	ECG (250 Hz)	Education only; battery power only with electrodes on a person



Wiring. I²C SDA GPIO21 / SCL GPIO22; AD8232 GPIO34 / 32 / 33; MMA8452Q INT1 GPIO27 (reserved); SPI 18/19/23 reserved. All modules on 3.3 V.

Read: `docs/firmware.md`, `docs/hardware.md`.

10 Bluetooth protocol

One GATT service (`57570000-5757-4b57-8000-574541525753`) with INFO, CAPS, CONTROL, DATA, WIFI and TLS characteristics, all requiring an encrypted, PIN-authenticated link (LE Secure Connections). Data packets are a 58-byte header plus int16 samples plus a CRC-32, at most 462 bytes. Wire codes 1–6 never change meaning. Optional Wi-Fi streaming uses TLS with the patch's certificate pinned over the authenticated Bluetooth link.

Read: `docs/ble-protocol.md`.

11 Backend, accounts and security

- **Supabase:** Postgres 17 with Row Level Security, Auth (email and password), a private Storage bucket `recordings`, and Realtime (postgres changes for alerts and falls; private broadcast for live view). Nine tables, about 15 RPCs and a sign-up trigger, all created by `supabase/migrations/`. `supabase/seed.sql` adds synthetic demo data **locally only**.
- **Accounts:** 10-character minimum password on the server. Sign-up must declare consent, 18+ and a known terms version, or the server refuses it. Email links return through `wearwise://auth/*` (PKCE). Researchers and clinicians are created by an administrator; nobody changes their own role.
- **Access rules:** every table has RLS and least-privilege grants; anonymous users can do nothing; participants own their rows; viewers read only what a live grant covers; writes need an active, 18+-confirmed account. Verified by 277 SQL checks plus pgTAP and an API-level two-user test.

Read: `docs/database.md`, `docs/security.md`, and the Security Documentation PDF.

PART 3

Running and releasing

Local development, cloud deployment, administration, and release builds.

12 Local development

Terminal (repository root)

```
$ npm install
$ supabase start                # Docker running; applies migrations and demo data
$ npm run backend:local         # point both apps at the local backend
$ npm run dev -w dashboard      # http://localhost:5192
$ cd apps/mobile && npx expo run:android
```

Day to day: `supabase db reset` rebuilds the database; `npm run backend:status` shows which backend each app uses; `npx expo start -c` restarts the bundler after a configuration change. Demo accounts (local only, password `wearwise-demo-only`): `demo.participant@example.invalid`, `demo.other@example.invalid`, `demo.researcher@example.invalid`.

Step by step for people new to development: the **Setup Guide** PDF.

13 Tests

What	Command (repository root)
Phone app: types and unit tests	`npm run typecheck -w mobile` · `npm test -w mobile`
Dashboard and legal package	`npm run typecheck -w dashboard` · `npm test -w dashboard`
Database security (resets the local database)	`bash supabase/sql-tests/run-on-local-stack.sh`

What	Command (repository root)
Database: pgTAP only	`supabase test db`
Hosted database matches the migrations	`supabase/admin/schema-fingerprint.sql` (read-only)
Hardware	`docs/testing/hardware-test-matrix.md`

14 Administration

Day-to-day account tasks need no coding: they are SQL scripts in `supabase/admin/`, run in the Supabase SQL Editor.

Task	Script
Add a researcher or clinician	`create-viewer.sql` (one-time password, sent privately). Illustrated in the **Adding a Researcher or Clinician** PDF
Promote an existing account	`set-role.sql`
Locked out	`reset-password.sql`
Suspend or reactivate	`set-status.sql`
Who has access	`overview.sql` (read-only)
Delete an account and its data	`node supabase/scripts/admin.mjs delete-account`

Runbook: `docs/operations/researcher-access.md`.

15 Cloud deployment

1. Create a project under the organisation's Supabase account (or use a self-hosted Supabase over HTTPS).
2. `supabase login`, `supabase link --project-ref <ref>`, `supabase db push`. Never push the demo seed; never run `supabase config push` with the local configuration.
3. Settings: email confirmation on, own SMTP, Site URL and redirect URLs for the hosted dashboard, Realtime "Allow public access" off.
4. Create the first researcher with `create-viewer.sql`.
5. Put the project URL and publishable key in `apps/mobile/.env`, `.env.production` and `apps/dashboard/.env`; `npm run backend:cloud`.
6. Host the dashboard: `npm run deploy-config -w dashboard` (security headers for your backend), `npm run build -w dashboard`, and serve `apps/dashboard/dist/` on HTTPS with single-page fallback.

Read: `docs/deployment.md`. The original capstone project's database differs from the migrations only by one comment; to keep it instead, see "Taking over the original capstone project" there.

16 Release builds

Android	JDK 17, Android SDK 36. `.env.production` with an https backend, your own keystore (`node scripts/create-release-signing.cjs` in `apps/mobile`, then back it up), `npx expo prebuild --platform android`, then `NODE_ENV=production ./gradlew assembleRelease` (or `bundleRelease`). The build refuses a non-HTTPS backend or a missing key. Check the APK with `tools/audit-apk.ps1`.
iOS	macOS, Xcode, CocoaPods. `node scripts/check-release-config.cjs`, `npx expo prebuild --platform ios`, open `ios/WearWise.xcworkspace`, choose your team (change the bundle identifier if taken), Archive.
Firmware	Bump `WW_FW_VERSION`, build from a clean checkout (`idf.py build`), then `npm run firmware:export -w dashboard`; the manifest records the commit.
Versions	App `version` / `versionCode` in `app.config.ts`; dashboard in its `package.json`.

PART 4

Changing WearWise

Common changes, step by step, and how to keep the system healthy.

17 Adding a sensor or a database field

A new sensor

1. **Firmware:** a driver implementing `WearWiseSensor.h`, registered in `SensorManager.c` behind a `WW_ENABLE_*` flag.
2. **New kind of measurement only:** the next free wire code (7 or 8) with scale and unit in both `communication/protocol.{c,h}` and `packages/sensor-protocol/src/types.ts`; regenerate fixtures (`npm run fixtures -w @wearwise/sensor-protocol`).
3. **App and dashboard:** labels, units, ranges (`apps/mobile/src/domain/vitals/`), charts.
4. **Backend:** widen the sensor-code checks in a **new migration**.
5. **Tests:** firmware boot self-test, app tests, SQL suites, and a hardware-matrix row.

A new database field or table

1. `supabase migration new <name>`. A new table needs RLS, explicit grants to `authenticated`, policies for each operation, and indexes on foreign keys.
2. `supabase db reset`; add checks to a `supabase/sql-tests/*.local.sql` suite; run the suites.

3. Update the app's sync gateway and types, and the dashboard's queries (`apps/dashboard/src/data/`).
4. Document it in `docs/database.md`; on the hosted project, `supabase db push`. Never edit an applied migration: add a new one.

18 Maintenance and responsibilities

How often	Do
Every change	Unit tests (app, dashboard); database suites if SQL changed
Every release	Bump versions; run the hardware-matrix rows touched; export the firmware if it changed
Every few months	Update the Expo SDK (this also clears the remaining dependency alerts), React Native, Supabase CLI and images, ESP-IDF; rerun everything, including the handover dry run
When the terms change	New `ACCEPTANCE_VERSION` in `packages/legal` plus a migration adding it to `private.terms_versions`, applied before the apps ship
When people leave	Suspend or delete their accounts; rotate any credential they held

Secrets the owner creates and keeps

The Supabase project and database password, service keys, personal access tokens, SMTP credentials, the Android release keystore and Apple signing. **None are in the repository**, and none should ever be. The project URL and publishable key ship inside the apps and are public by design; Row Level Security protects the data.

Troubleshooting for every part: `docs/troubleshooting.md` and the Setup Guide, section 27.

PART 5

Limitations and next steps

What WearWise can't do yet, and where to take it.

19 Known limitations

- **Measurements:** research prototype, not clinically validated. SpO_2 is uncalibrated (a board read about 80 % at rest). Heart rate can read about double before settling, or on a weak module.
- **Falls:** possible-fall detection is a threshold heuristic; nothing outside WearWise is alerted.
- **Testing coverage:** emulators have no Bluetooth; iOS has had less real-device testing than Android, including the pinned Wi-Fi stream.
- **Patches:** one phone connected at a time; a patch remembers two devices; the label PIN is fixed until changed with *New PIN*; firmware updates are USB-only; flash is unencrypted.
- **Accounts and data:** no two-factor sign-in for staff; no in-app retention or purge; no audit log of researcher reads.
- **Paperwork:** the legal documents are drafts until the organisation's details are filled in.

- **Dependencies:** five advisories remain, all in build or test tooling or mitigated; they clear with an Expo SDK upgrade.
- **Cosmetic:** Home's "Backed up x/y" counts a recording in progress as backed up until it stops.

20 Future improvements

- ✓ Calibrate SpO_2 against a reference pulse oximeter; validate heart rate against a reference (the AD8232 on the same board is a good start: `docs/HRValidation.md`).
- ✓ Hold back heart rate until the estimate has settled.
- ✓ Emergency-contact notification for falls, and interrupt-driven motion (MMA8452Q INT1 on GPIO27).
- ✓ More sensors through the driver interface; an SPI sensor on the reserved pins.
- ✓ Two-factor sign-in for staff; a column-limited view for viewers; read audit logging.
- ✓ Retention controls and data export for participants.
- ✓ Patch hardening (Secure Boot, flash encryption) with signed over-the-air firmware updates.
- ✓ Continuous integration running the app, dashboard and database suites on every change, with secret scanning.
- ✓ An independent security review before any wider use.

A Documentation map

Document	For
PDF WearWise Handover Documentation (this document)	The handover at a glance
PDF `docs/guides/WearWise-User-Guide.pdf`	Every screen of the app and the dashboard
PDF `docs/guides/WearWise-Setup-Guide.pdf`	Setting WearWise up, for people new to development
PDF `docs/guides/WearWise-Adding-a-Researcher.pdf`	Administrators: researcher and clinician accounts
PDF `docs/security/WearWise-Security-Documentation.pdf`	Controls, threat model, risks, owner responsibilities
`README.md`	Technical quick start and runbook
`docs/knowledge-transfer.md`	The handover guide in text form
`docs/architecture.md`, `docs/mobile-app.md`	System design; app internals
`docs/firmware.md`, `docs/hardware.md`, `docs/ble-protocol.md`	The patch
`docs/database.md`, `docs/deployment.md`	Schema, RLS and tests; cloud, self-hosted and release builds
`docs/security.md`, `docs/security/`	Security notes, audits, dependency assessment
`docs/operations/researcher-access.md`	Admin runbook
`docs/troubleshooting.md`	When something doesn't work

Document	For
`docs/testing/`	Test strategy, hardware matrix, real-device and dry-run reports
`TEAM.md`, `LICENSE`, `NOTICE`	Team, licence, third-party components

B Repository layout

Repository

```
apps/mobile/           Phone app: React Native + Expo SDK 57 (Android, iOS)
apps/dashboard/        Web dashboard: React + Vite (also Patch setup over USB)
packages/              Shared TypeScript: sensor-protocol, legal, password, design-tokens
firmware/              ESP32 firmware (ESP-IDF 5.4): wearwise-esp32/
supabase/              Backend: config.toml, migrations/, seed, tests, sql-tests, admin, scripts
scripts/               use-backend.mjs (switch the apps between local and cloud)
docs/                  Documentation, guides (PDF + sources), security, testing
design-system/tokens/  Brand design tokens (CSS)
tools/audit-apk.ps1    Checks a release APK for leaked secrets or developer paths
```

Made by the WearWise Project Team

Zakiul Atheef	Project Lead, Scrum Master, Technical Lead, Security Engineering Lead and Integration Lead
Hari Bethelli	Cloud & Data Engineer
Siri Bethelli	UI/UX & Data Visualisation Engineer
Thevindu Kodagoda Munasinghage	Security Engineer, Documentation Lead and QA & Testing Lead

Project Owner: Shanmuga Sundar Dhanabalan, Deputy Director, CTI, La Trobe University (project brief and stakeholder direction). **Development:** the WearWise Project Team, the student development team. © 2026 WearWise Project Team · La Trobe University CSE3CAP Capstone Project. Licensed under the PolyForm Noncommercial License 1.0.0: free to use, copy and change for noncommercial purposes, not for sale. See `LICENSE`, `NOTICE` and `TEAM.md` in the repository.

WearWise Handover Documentation · handover release v1.0.0, October 2026. WearWise is a research prototype and not a medical device. Its security practices are informed by relevant guidance; nothing about WearWise is certified or independently tested.